

MalOps: Designing and Evaluating a Reproducible MLOps Pipeline for Malaria Risk Prediction

Naa Lamiorkor Boye and Likhita Kolli

Lucerne School of Computer Science and Information Technology

Lucerne University of Applied Sciences and Arts (HSLU), Rotkreuz, Switzerland

Emails: {naalamiorkor.boy, likhita.kolli}@stud.hslu.ch

GitHub Repository: github.com/Lamiorkor/Malaria_Outbreak_Prediction

Abstract—This paper presents MalOps, an end-to-end Machine Learning Operations (MLOps) prototype for predicting whether a country-year will be followed by a year of high malaria incidence. The project integrates temporal feature engineering, a class-balanced Logistic Regression model, MLflow experiment tracking, FastAPI serving, Docker and Docker Compose, PostgreSQL logging, Grafana dashboards, Prefect scheduling, pytest-based testing, and GitHub Actions continuous integration and delivery. The prepared panel contains 2,402 country-year records for 106 countries from 2000–2022; after next-year label construction and lag requirements, 1,711 records across 100 countries remain. On a strict temporal split, the saved model achieves validation AUC-ROC 0.993 and test AUC-ROC 0.998. Test recall is 1.000, with an exact 95% confidence interval of 0.918–1.000, and test precision is 0.878. However, a persistence rule based only on current malaria incidence achieves test AUC-ROC 1.000 and F1 0.977, revealing that the operational target is dominated by incidence persistence. This finding changes the interpretation of the project: the principal contribution is the reproducible lifecycle and the lessons learned about target design, train-serve consistency, monitoring, testing, and automation, rather than a novel predictive model. The implementation is a local prototype, not a prospectively validated public-health surveillance system.

Index Terms—MLOps, malaria, high-incidence risk, temporal validation, data drift, FastAPI, Docker, MLflow, Grafana, Prefect, CI/CD.

I. INTRODUCTION

Malaria remains a major public-health challenge. The World Health Organization estimated 282 million cases and 610,000 deaths in 2024, with the WHO African Region carrying most of the burden [1]. Temperature and rainfall influence mosquito and parasite biology, but their effects are nonlinear and context dependent rather than uniformly positive [2], [3]. These properties make malaria risk a plausible use case for data-driven early warning, while also making models vulnerable to changing climatic, socioeconomic, and surveillance conditions.

A predictive model alone is not an operational system. Production-oriented machine learning also requires reproducible data preparation, artifact tracking, serving, input validation, logging, testing, deployment automation, and monitoring. Sculley *et al.* describe the resulting hidden technical debt in ML systems [4]; Breck *et al.* similarly frame production readiness in terms of tests and monitoring rather than accuracy alone [5]. These ideas align with the course objective of

training, deploying, monitoring, and applying CI/CD to a machine-learning model.

MalOps was developed as a project-scale implementation of that complete lifecycle. It predicts a project-defined next-year high-incidence label using malaria incidence, precipitation, population density, GDP per capita, and temperature variables. The repository implements feature engineering, temporal evaluation, model and scaler persistence, MLflow tracking, FastAPI inference, PostgreSQL logging, Docker-based local deployment, Grafana monitoring, Prefect scheduling, and GitHub Actions automation [6].

The revised report makes three deliberate distinctions. First, the target is an *operational high-incidence proxy*, not an official epidemiological outbreak definition. Second, continuous delivery of a Docker image to GitHub Container Registry (GHCR) is not the same as automatic cloud deployment. Third, scheduled execution is not continual learning unless monitoring, retraining, evaluation, approval, and model replacement form a controlled feedback loop.

The main contributions are:

- an auditable temporal modeling pipeline with 22 engineered features and train-only scaling;
- a quantitative re-evaluation that includes confidence intervals, calibration, simple baselines, and feature-family ablations;
- a local end-to-end MLOps architecture connecting training, tracking, serving, persistence, monitoring, orchestration, and CI/CD; and
- a critical maturity assessment identifying what was implemented, what remains partial, and what was learned from the operationalization process.

II. RELATED WORK AND DESIGN CONTEXT

A. Malaria, Climate, and Predictive Modeling

Malaria transmission depends on interactions among vector ecology, parasite development, human vulnerability, interventions, and surveillance. Mechanistic work estimates a thermal optimum near 25–26 °C and declining suitability at higher temperatures, which motivates nonlinear temperature features rather than a simple monotonic assumption [2], [3]. The present project does not attempt a mechanistic epidemiological model; it uses annual country-level variables as predictors in a retrospective classification exercise.

B. Evaluation in High-Stakes Prediction

Responsible health prediction requires more than discrimination. Wiens *et al.* emphasize deployment readiness and avoidance of harm [7], while WHO guidance stresses transparency, accountability, inclusiveness, and human oversight for AI in health [8]. Because the positive class is imbalanced, precision-recall analysis complements ROC analysis [9]. Probability calibration is also necessary when scores may inform resource allocation [10]. Finally, temporal and geographical dependence can make random validation optimistic; blocked validation is preferable when the intended use involves extrapolation across time or space [11].

C. MLOps and Distribution Shift

MLOps treats code, data, configuration, and learned artifacts as interacting system components [12]. Distribution shift can cause models to fail silently, motivating explicit monitoring [13]. Concept drift literature further distinguishes changes in input distributions from changes in the relationship between predictors and outcomes [14]. MalOps implements a limited form of feature-distribution monitoring using two-sample Kolmogorov–Smirnov (KS) tests; it does not claim full concept-drift diagnosis.

III. DATA, TARGET, AND EXPERIMENTAL DESIGN

A. Prepared Dataset and Data Lineage

The repository contains a merged CSV with 2,402 rows, 106 countries, and years 2000–2022. It has nine columns: country name and code, year, malaria incidence, precipitation, population density, GDP per capita, annual mean temperature, and growing-season mean temperature. No missing values occur in the prepared file. The malaria-incidence variable is expressed per 1,000 population at risk, consistent with World Development Indicator SH.MLR.INCD.P3 [15]; population density and GDP per capita align with standard World Bank indicators [16], [17].

The repository does not retain immutable raw-source snapshots, download timestamps, or source-fetching scripts for every climate variable. Therefore, the report cites the prepared project dataset as the direct experimental artifact and treats incomplete upstream provenance as a reproducibility limitation. A mature pipeline should version raw data and document exact indicator codes, transformation steps, licenses, and retrieval dates.

B. Operational Target Definition

For country c and feature year t , the label is

$$y_{c,t} = \mathbb{I}(I_{c,t+1} \geq Q_{0.75}^{\text{train}}), \quad (1)$$

where $I_{c,t+1}$ is next-year malaria incidence and $Q_{0.75}^{\text{train}} = 214.335$ is the 75th percentile of incidence computed from years no later than 2016. Computing the threshold only from the training period avoids direct leakage from validation and test years.

This label should be described as *next-year high-incidence risk*. It is not a country-specific anomaly threshold and is

TABLE I
DATASET AND FEATURE-STORE AUDIT

Item	Observed value
Prepared panel	2,402 rows; 106 countries; 2000–2022; 0 missing cells
Model-ready panel	1,711 rows; 100 countries; feature years 2003–2021
Target	Indicator that incidence in year $t + 1$ is at least 214.335
Train	1,300 rows; 342 positives; 2003–2016
Validation	252 rows; 72 positives; 2017–2019
Test	159 rows; 43 positives; 2020–2021
Predictors	22 numeric features

not an official WHO outbreak definition. A country with persistently high incidence may remain positive every year, while a sharp increase in a low-incidence country may remain below the global threshold. In the model-ready panel, 68 of 100 countries are never positive, 18 are positive in every eligible year, and only 14 change class. Figure 1 visualizes this persistence and the temporal split.

C. Temporal Feature Engineering

Features are computed within country groups after sorting by country and year. The final set contains six base variables, six lags, three rolling means, and seven derived features. The fitted `StandardScaler` uses training data only and is serialized with the model.

The notebook also exports SMOTE-resampled data, but the production training script does not use it. The final model instead trains on the original scaled training rows with `class_weight="balanced"`. This distinction is important for artifact lineage: exploratory outputs should not be confused with the deployed training path.

D. Evaluation Protocol

The strict temporal split mirrors a future-prediction setting: training ends in 2016, validation uses 2017–2019, and testing uses 2020–2021. Evaluation reports AUC-ROC, average precision, precision, recall, F1, accuracy, and Brier score. Stratified bootstrap intervals with 3,000 resamples are used for most metrics; because recall has no observed false negatives, its interval is computed with an exact Clopper–Pearson method. The evaluation additionally includes:

- a majority-class baseline;
- a persistence rule predicting positive when current incidence is at least 214.335;
- a one-feature Logistic Regression using current incidence only; and
- ablations using epidemiological-history, climate-only, socioeconomic-only, and combined climate-socioeconomic feature groups.

IV. MODEL RESULTS

A. Saved Production Model

The saved artifact is a Logistic Regression model implemented with scikit-learn [18]:

Operational target and temporal structure

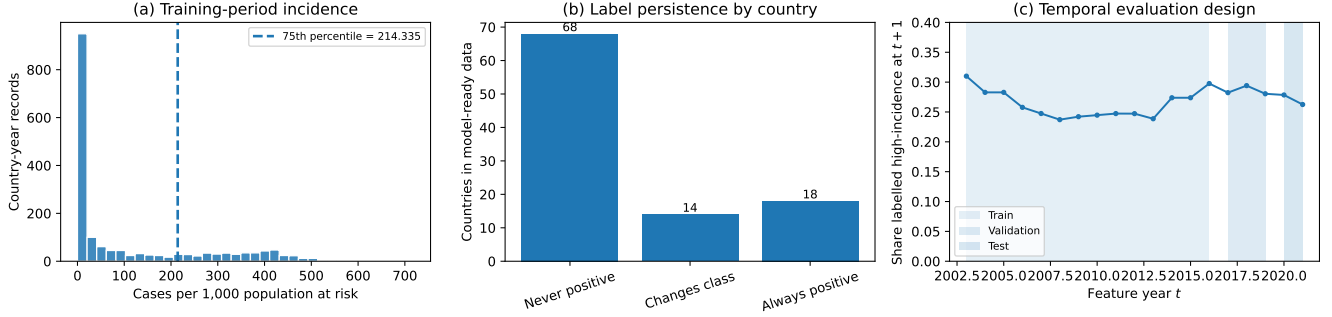


Fig. 1. Operational target audit. The global training-period threshold creates a highly persistent country-level label, which explains why current incidence alone is a strong baseline.

TABLE II
FEATURE FAMILIES USED BY THE FINAL MODEL

Family	Count	Features
Base	6	Malaria incidence, precipitation, population density, GDP per capita, annual temperature, growing-season temperature
Lags	6	Malaria lags 1–3; one-year temperature, precipitation, and GDP lags
Rolling	3	Three-year malaria, temperature, and precipitation means
Derived	7	Malaria year-over-year change, temperature–precipitation interaction, temperature squared, climate-risk index, vulnerability index, log GDP, log malaria incidence

TABLE III
SAVED MODEL PERFORMANCE WITH 95% INTERVALS

Metric	Validation	Test
AUC-ROC	.993 [.981, 1.000]	.998 [.992, 1.000]
Average precision	.954 [.879, 1.000]	.993 [.975, 1.000]
F1	.980 [.954, 1.000]	.935 [.887, .977]
Precision	.960 [.911, 1.000]	.878 [.796, .956]
Recall	1.000 [.950, 1.000]	1.000 [.918, 1.000]
Accuracy	.988 [.972, 1.000]	.962 [.931, .987]
Brier score	.013 [.004, .026]	.027 [.011, .047]

```
LogisticRegression(
    C=1.0, class_weight="balanced",
    solver="liblinear", max_iter=1000,
    random_state=42)
```

At threshold 0.50, the validation confusion matrix contains 177 true negatives, 3 false positives, 72 true positives, and no false negatives. The test confusion matrix contains 110 true negatives, 6 false positives, 43 true positives, and no false negatives.

Figure 2 shows discrimination, precision-recall behavior, calibration, and the test confusion matrix. The test Brier score of 0.027 is low, but the small evaluation window and near-binary probability distribution limit strong claims about calibration. A default threshold of 0.50 was used; it was not optimized from an explicit public-health cost function.

B. Baseline and Feature-Family Audit

Table IV is the most important qualification of the headline AUC. The persistence rule outperforms the full model on

test F1, precision, accuracy, and Brier score. Current incidence alone yields perfect test ranking. Climate-only and socioeconomic-only models contain signal, but are much weaker.

The result does not invalidate the MLOps implementation, but it changes the scientific interpretation. The model mainly learns persistence in a global high-incidence state. It does not demonstrate that climate features materially improve one-year-ahead outbreak detection. Future work should compare alternative targets, such as country-specific exceedances, year-over-year surges, or epidemiologically validated outbreak labels.

V. MLOPS ARCHITECTURE AND IMPLEMENTATION

Figure 3 distinguishes the implemented data/model path from automation and observability. The architecture is local and reproducible rather than cloud-native.

A. Experiment Tracking and Artifact Handoff

The training script loads the scaled temporal splits, fits the final model, logs parameters and metrics to MLflow, logs a model artifact, and saves `logistic_regression.pkl`, `scaler.pkl`, and `model_metadata.json`. MLflow is used for experiment traceability [19]. The metadata stores the 22-feature order, threshold, temporal windows, parameters, and evaluation metrics.

The API does not load the active MLflow registry stage at startup. It loads local serialized artifacts copied into the Docker image. This is a legitimate self-contained deployment pattern, but the code comments and system diagram must describe it accurately. Registry-based promotion remains a future enhancement.

Retrospective validation of the saved production model

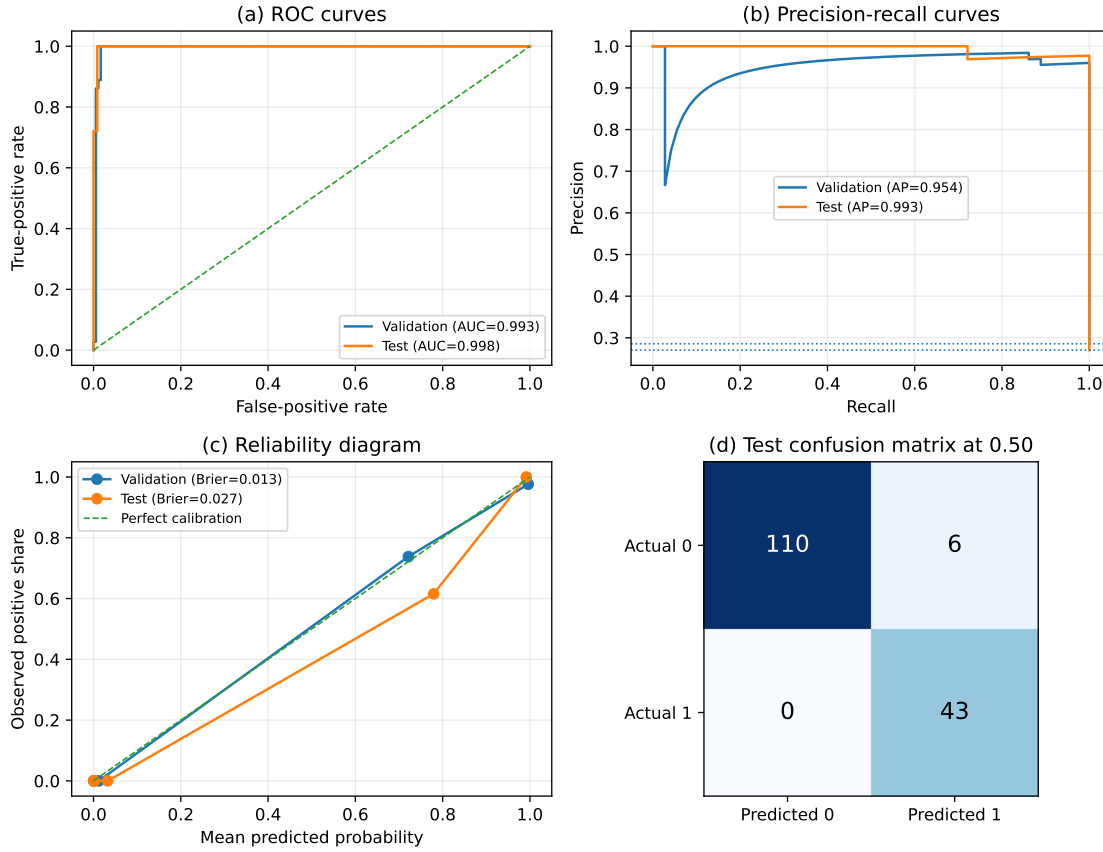


Fig. 2. Retrospective validation of the saved production model. Precision-recall and calibration are shown alongside ROC curves because AUC-ROC alone is insufficient for imbalanced and decision-oriented prediction.

TABLE IV
TEST-SET BASELINES AND FEATURE-FAMILY ABLATIONS

Model or feature set	AUC	AP	F1	Prec.	Recall	Brier
Persistence rule	1.000	1.000	.977	.956	1.000	.013
Current incidence only	1.000	1.000	.915	.843	1.000	.026
Epidemiological history	1.000	.999	.915	.843	1.000	.028
Full 22-feature model	.998	.993	.935	.878	1.000	.027
Climate + socioeconomic	.804	.597	.569	.470	.721	.186
Socioeconomic only	.784	.555	.604	.508	.744	.182
Climate only	.629	.346	.448	.330	.698	.253
Majority-class baseline	.500	.270	.000	.000	.000	.270

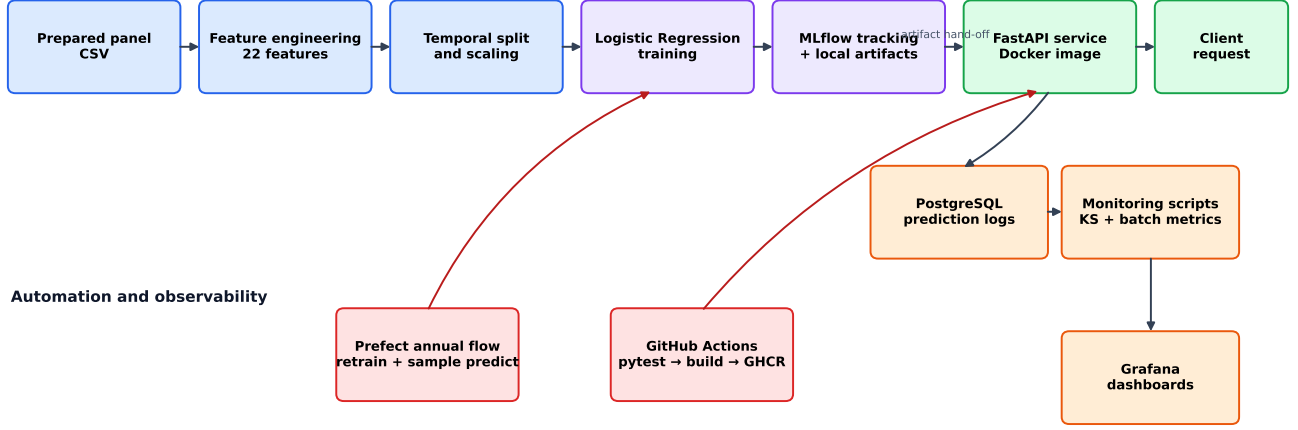
B. Serving Consistency and Resilient Telemetry

FastAPI exposes `GET /`, `GET /model/info`, and `POST /predict` [20]. Pydantic validates basic input bounds, while the prediction pipeline reconstructs the 22-feature vector from the current country-year record and up to three years of history. The response contains the predicted probability, binary alert, risk tier, threshold, model name, timestamp, and inference latency.

The implementation revealed a train-serve skew risk. During offline feature preparation, observations with undefined lag

or rolling values are removed before model fitting. During online serving, however, undefined engineered values are filled with zero. Consequently, the same logical country-year can be represented differently during training and inference. The preferred correction at the present project scale is to extract all transformations into a versioned, immutable Python package, for example `malops_features`. Both the training pipeline and FastAPI service would import the same functions, feature order, missing-history policy, schema version, and validation rules. The package version would be recorded in MLflow, the

Implemented data and model path



Scope boundary: local Docker Compose prototype; serving loads versioned local artifacts rather than the MLflow registry; drift does not automatically promote a model.

Fig. 3. Implemented MLOps architecture and scope boundary. The serving service loads versioned local artifacts; MLflow records training runs, while GitHub Actions delivers Docker images to GHCR.

model metadata, and the Docker image. A parity test would then generate offline and serving features for identical records and fail CI whenever the two vectors differ.

At a larger operational scale, a feature store such as Feast could centralize entities and feature views for point-in-time historical retrieval and online serving [21]. A feature store would not remove the need for well-defined transformations, but it would reduce duplicate data paths and enforce one registered feature schema. For this project, the simpler shared-package design is proportionate and could later be migrated to a feature-store architecture.

Prediction telemetry is currently written synchronously to PostgreSQL. This provides an audit trail, but database downtime or network latency can cause an otherwise valid prediction request to fail. A resilient design would separate inference from persistence. For a lightweight deployment, FastAPI BackgroundTasks could return the prediction before attempting the database insert. For stronger delivery guarantees, the API would publish a compact prediction event to RabbitMQ or Redis, and a Celery worker would persist it asynchronously with retries, idempotency keys, and a dead-letter queue [22]. Queue depth, failed writes, and processing delay would become operational metrics. The prediction endpoint would therefore remain available even when the monitoring database is temporarily unavailable.

C. Containerization and CI/CD

Docker packages the API and model artifacts into a reproducible runtime [23]. The root Docker Compose file starts the API, MLflow, and PostgreSQL; a separate monitoring Compose file starts PostgreSQL, Adminer, and Grafana for monitoring demonstrations.

GitHub Actions implements two gated jobs [24]. The CI job performs syntax checks and executes 20 pytest tests. The

TABLE V
AUTOMATED TEST COVERAGE

Suite	Tests	Primary purpose
Core unit tests	7	Numeric input, length, shape, validation order, error handling
API contract tests	9	Required fields, types, optional history, and value bounds
Pipeline smoke tests	4	Feature reconstruction, probability bounds, threshold logic, response schema
Container smoke test	–	Health endpoint after image build and pull

CD job runs only after CI succeeds, builds the Docker image, publishes branch, commit-SHA, and latest tags to GHCR, and smoke-tests the resulting container. Operational evidence for the hosted CI/CD run and GHCR image publication is provided in Appendix A. This keeps the main body focused on the pipeline design while retaining implementation proof.

D. Prefect Scheduling and Proposed Model-Promotion Gate

The implemented Prefect flow runs the training script and then generates a sample Ghana prediction. The deployment is named `malaria-yearly` and uses cron expression `0 9 5 12 *`, corresponding to 09:00 on December 5 each year. Prefect provides orchestration, retries, scheduling, and run visibility [25]. The current flow nevertheless demonstrates scheduled automation rather than continual learning because it does not ingest new data, compare models through a formal gate, or change the artifact used by the API.

A controlled continual-learning extension would implement the following champion–challenger workflow:

- 1) ingest newly released malaria, climate, population, and GDP records and store an immutable data-version iden-

tifier;

- 2) construct features using the same shared transformation package used by the serving service;
- 3) train and register a challenger model in MLflow, together with its feature-schema, data-version, dependency, and calibration metadata;
- 4) load the model referenced by the MLflow `champion` alias and evaluate both models on the same rolling out-of-time validation window;
- 5) bootstrap the paired prediction results and calculate the confidence interval for $\Delta F_1 = F_{1,\text{challenger}} - F_{1,\text{champion}}$; and
- 6) promote the challenger only when all quality, performance, and governance gates pass.

One possible algorithmic gate is

$$G = [\text{LB}_{95\%}(\Delta F_1) > 0] \wedge [R_c \geq R_p - 0.01] \wedge [B_c \leq B_p + 0.01] \wedge [\text{schema and integration tests pass}], \quad (2)$$

where R denotes recall, B denotes Brier score, and subscripts c and p denote challenger and production models. If G is false, the existing champion remains unchanged and the flow records the rejection and notifies the maintainers. If G is true, the challenger may first receive a `candidate` alias and undergo shadow or human review before the `champion` alias is reassigned. MLflow aliases allow the serving service to reference a stable name while the underlying model version changes [26].

Because the use case concerns public-health prioritization, automatic metric-based promotion alone would be insufficient. A production design should retain a documented human approval and rollback step, particularly when the new model changes country-level risk classifications.

VI. MONITORING AND OPERATIONAL EVIDENCE

A. Reference Data and Natural Versus Synthetic Drift

The monitoring reference is created from 2017–2019 records passed through the serving path. It contains 317 rows, compared with 252 validation rows in the model-ready feature store. This difference exposed the train-serve inconsistency discussed previously: offline preparation removes records with undefined derived values, whereas the serving path fills missing engineered values with zero.

The batch generator samples records after the reference period that contain three years of history. It supports `none`, `climate`, `economic`, `mixed`, and `extreme` modes. The latter four modify selected feature distributions to create repeatable monitoring demonstrations.

Synthetic drift was ultimately not necessary to establish that the historical data changed. Exploratory comparisons between the reference window and later unmodified records already detected shifts in several epidemiological, socioeconomic, and temperature variables. In addition, a stored `none` batch still flagged one of the six monitored features. The latter observation alone could be caused by sampling variation, but together the results indicate that organic temporal drift was

already present in the dataset. Artificial injection was therefore most useful as a controlled stress test that produced repeatable dashboard scenarios, rather than as evidence that drift exists.

B. Statistical Critique and Improved Drift Policy

The implemented script applies an independent two-sample Kolmogorov–Smirnov test to six raw serving features and declares drift when $p < 0.05$. This approach is easy to explain, but it has three important statistical weaknesses.

First, repeated univariate testing inflates the probability of at least one false alarm. Under independence, six tests at $\alpha = 0.05$ yield a family-wise false-positive probability of approximately $1 - 0.95^6 = 0.265$. Extending the same rule to all 22 model features would increase it to approximately $1 - 0.95^{22} = 0.676$. A Bonferroni-controlled threshold would instead use

$$\alpha^* = \frac{0.05}{m}, \quad (3)$$

which equals 0.0083 for six tests and approximately 0.0023 for 22 tests [27]. A Holm procedure could later provide a less conservative alternative.

Second, a KS p-value does not measure the operational size of a shift. Small batches have weak and unstable detection power, while very large batches can make minor differences statistically significant. The pipeline should therefore retain both the KS statistic and adjusted p-value and pair them with an effect-size measure. Population Stability Index (PSI) provides a bin-based, interpretable summary, although its result depends on the selected bins [28]. A normalized Wasserstein distance provides a continuous measure of how far one empirical distribution must move to match another [29]. Thresholds for either measure should be calibrated from historical backtests rather than treated as universal constants.

Third, feature drift is not equivalent to performance degradation. An improved policy would distinguish:

- 1) schema, missingness, and range violations;
- 2) adjusted statistical evidence of feature shift;
- 3) effect-size and persistence of the shift across consecutive batches;
- 4) changes in the prediction distribution; and
- 5) delayed performance decay once reliable outcome labels arrive.

A warning should be raised only when the adjusted significance threshold and an effect-size threshold are both exceeded, or when the same moderate shift persists across multiple batches. Retraining should not be triggered solely because one feature has a small p-value.

C. Metrics, Storage, and Dashboard Evidence

For each batch, the current monitoring script stores batch size, drift mode, number and share of drifted features, accuracy, precision, recall, macro F1, mean probability, risk-share distribution, mean latency, and selected feature means in `monitoring_metrics`. Per-record predictions are stored in `prediction_logs`. Grafana queries these tables to visualize drift, classification metrics, probability trends, input trends, prediction volume, and country-level risk [30].

A crucial implementation detail is that `calculate_metrics.py` deliberately flips predictions with probabilities between 0.4 and 0.6 to simulate realistic mistakes. Consequently, the stored dashboard classification metrics demonstrate the monitoring pipeline but are not independent estimates of deployed model performance.

The results illustrate why a single p-value rule is insufficient. The unmodified batch identifies drift, while one mixed synthetic batch identifies none. Both sampling variability and batch size affect the tests. The evidence also confirms that controlled injection should be described as a demonstration facility; naturally occurring historical shift was already observable.

D. Operational Boundary of the Monitoring Prototype

The implementation provides feature-shift checks, batch metrics, persistent telemetry, and dashboards. It does not yet implement corrected multi-feature testing, PSI or Wasserstein thresholds, label-shift analysis, alert routing, or drift-triggered model promotion. More importantly, retrospective outcome labels are currently available because historical data are being replayed. In a live setting, true next-year incidence would arrive with a substantial delay.

A mature monitoring architecture would therefore maintain separate alert classes for data quality, input shift, prediction shift, infrastructure health, and delayed model performance. Only persistent or materially important alerts would enter the Prefect retraining flow, and any challenger would still be required to pass the approval gate described in the previous section.

VII. LESSONS LEARNED AND MATURITY ASSESSMENT

A. Lifecycle Lessons

Table VII summarizes the main learning at each lifecycle stage. The most consequential lesson is that target design dominated algorithm choice. High AUC initially appeared to demonstrate sophisticated forecasting; the persistence baseline showed that the label largely represents a stable high-incidence state.

B. MLOps Maturity Assessment

C. Single Source of Truth and Environment Hardening

The audit exposed inconsistencies that should be resolved through a central configuration contract rather than corrected independently in each file. A compact `config.yaml` could define:

```
model_name: MalariaOutbreakPredictor
model_threshold: 0.50
risk_thresholds:
  medium: 0.50
  high: 0.70
feature_schema: feature_schema_v1.json
schema_version: 1
```

The training pipeline, prediction service, monitoring scripts, tests, and documentation generator would all load this configuration. The feature schema would contain the ordered list of 22 features and their types. Consequently,

`prediction_core.py` would obtain its expected feature count from the schema rather than retaining the outdated constant of 16. The API risk tiers and README table would be generated from the same threshold values, eliminating the current disagreement between LOW risk below 0.40 in the documentation and below 0.50 in the code.

The same principle applies to model loading. The `predict.py` docstring should state that local serialized artifacts are loaded, or the implementation should be changed to load the `MLflow champion` alias. Metadata should record the model version, feature schema, shared transformation-package version, Python version, scikit-learn version, and training-data identifier.

Runtime and development dependencies should also be separated. `requirements.txt` should contain only packages required to start the API, while `pytest`, `coverage`, and HTTP test clients should remain in `requirements-dev.txt`. A lock file or fully pinned requirements should be produced for the Docker build. This would remove `pytest` from the production image, reduce the attack surface and image size, and prevent the cross-version scikit-learn artifact warning encountered during implementation [31].

Finally, CI should add four contract checks: the metadata feature order must equal the scaler input order; the model input dimension must equal the schema length; documented risk thresholds must equal runtime thresholds; and offline and serving transformations must produce equal vectors for a fixed fixture. These checks turn documentation and configuration consistency into enforceable build conditions.

VIII. LIMITATIONS AND FUTURE WORK

The project remains a retrospective local prototype. Its principal limitations and next steps are:

- **Target and generalization:** the global percentile label represents persistent high incidence rather than an officially confirmed outbreak. Future work requires epidemiologically validated targets, regional or leave-country-out evaluation, and richer temporal data.
- **Data and feature consistency:** raw-source snapshots and complete lineage are not retained, while offline and serving missing values are handled differently. A versioned shared feature package and immutable data manifests should be implemented before further deployment.
- **Monitoring validity:** synthetic injection and altered predictions supported demonstration, although natural drift was already present. Corrected hypothesis testing, effect-size metrics, persistent-alert rules, and delayed labels are needed before drift can guide operational action.
- **Serving resilience:** database writes are synchronous. Asynchronous telemetry, queue monitoring, authentication, secrets management, and service-level objectives remain unimplemented.
- **Continual learning:** Prefect currently schedules retraining but does not govern model replacement. A champion–challenger comparison, statistical gate, human approval, model

TABLE VI
REPRESENTATIVE STORED MONITORING BATCHES

Mode	Size	Drifted	Share	Accuracy	Precision	Recall	F1	Latency (ms)
None	30	1	.167	1.000	1.000	1.000	1.000	6.57
Climate	30	3	.500	1.000	1.000	1.000	1.000	6.35
Mixed	60	0	.000	.967	.846	1.000	.948	6.43
Extreme	40	3	.500	.950	.778	1.000	.922	6.81

TABLE VII
LESSONS LEARNED ACROSS THE MLOPS LIFECYCLE

Stage	What was implemented	Primary lesson and proposed control
Data and target	Prepared panel, next-year label, country-grouped lags, temporal split	A leakage-safe target can still be conceptually weak. Persistence baselines and epidemiological target validation should precede model complexity.
Training and features	Balanced Logistic Regression, 22 features, saved scaler and MLflow runs	Model reproducibility depends on transformations and feature order. A versioned shared feature package, or later a feature store, should serve both offline and online paths.
Serving	FastAPI validation, feature reconstruction, probability and risk response	Zero-filling missing online features while dropping them offline creates train-serve skew. CI should compare feature vectors from both paths for identical records.
Telemetry	Synchronous PostgreSQL prediction and monitoring logs	Auditability must not reduce availability. Inference should publish an event to an asynchronous worker with retries and idempotent writes.
Monitoring	KS tests, natural and synthetic batches, PostgreSQL, and Grafana	Natural drift was already visible, so injection was mainly a repeatable stress test. Multiple testing, effect size, batch size, and delayed labels must be considered before triggering action.
Scheduling	Prefect annual deployment and completed runs	Scheduling is not continual learning. A challenger must outperform the champion on an out-of-time slice, pass statistical and safety gates, and remain reversible.
CI/CD and configuration	Twenty tests, image build, GHCR publication, and container smoke test	Automation exposes inconsistencies. Features, thresholds, schema versions, dependencies, and documentation should be derived from one configuration source.

TABLE VIII
CURRENT MLOPS CAPABILITY ASSESSMENT

Capability	Status	Evidence or boundary
Temporal training and evaluation	Implemented	Reproducible feature store and fixed train/validation/test windows
Experiment tracking	Implemented	MLflow runs, parameters, metrics, and model artifacts
Model artifact versioning	Partial	Files and metadata are version-controlled; no immutable data/model lineage manifest
API serving	Implemented	FastAPI health, model-info, and prediction endpoints
Containerization	Implemented	Docker image and local Compose stack
Prediction logging	Implemented	PostgreSQL prediction logs with model version and latency
Monitoring dashboards	Implemented	Stored batch metrics and Grafana dashboards
Scheduled execution	Implemented	Prefect annual deployment with completed runs
CI and container delivery	Implemented	Hosted tests, image build, GHCR tags, smoke test
Cloud deployment and rollback	Not implemented	No live environment, rollout strategy, health policy, or rollback automation
Automated retraining approval	Not implemented	No drift trigger, candidate comparison, human gate, or registry promotion
Security and governance	Partial	Input validation exists; secrets management, RBAC, audit policy, and model card remain incomplete

alias promotion, shadow validation, and rollback policy are required.

- **Environment reproducibility:** configuration, dependencies, feature schemas, and documentation should be generated from a single versioned source of truth.

IX. CONCLUSION

MalOps demonstrates a broad MLOps lifecycle around a country-year malaria prediction task: data preparation, temporal features, model training, MLflow tracking, FastAPI serving,

Docker packaging, PostgreSQL logging, Grafana monitoring, Prefect scheduling, automated testing, and CI/CD delivery. The saved Logistic Regression model has strong retrospective discrimination and no observed false negatives in the validation or test windows.

The deeper result is methodological. A simple persistence rule matches or exceeds the full model because the operational target is highly persistent. Natural distribution shift was also already visible in the historical data, showing that synthetic injection was not required to demonstrate drift, although it

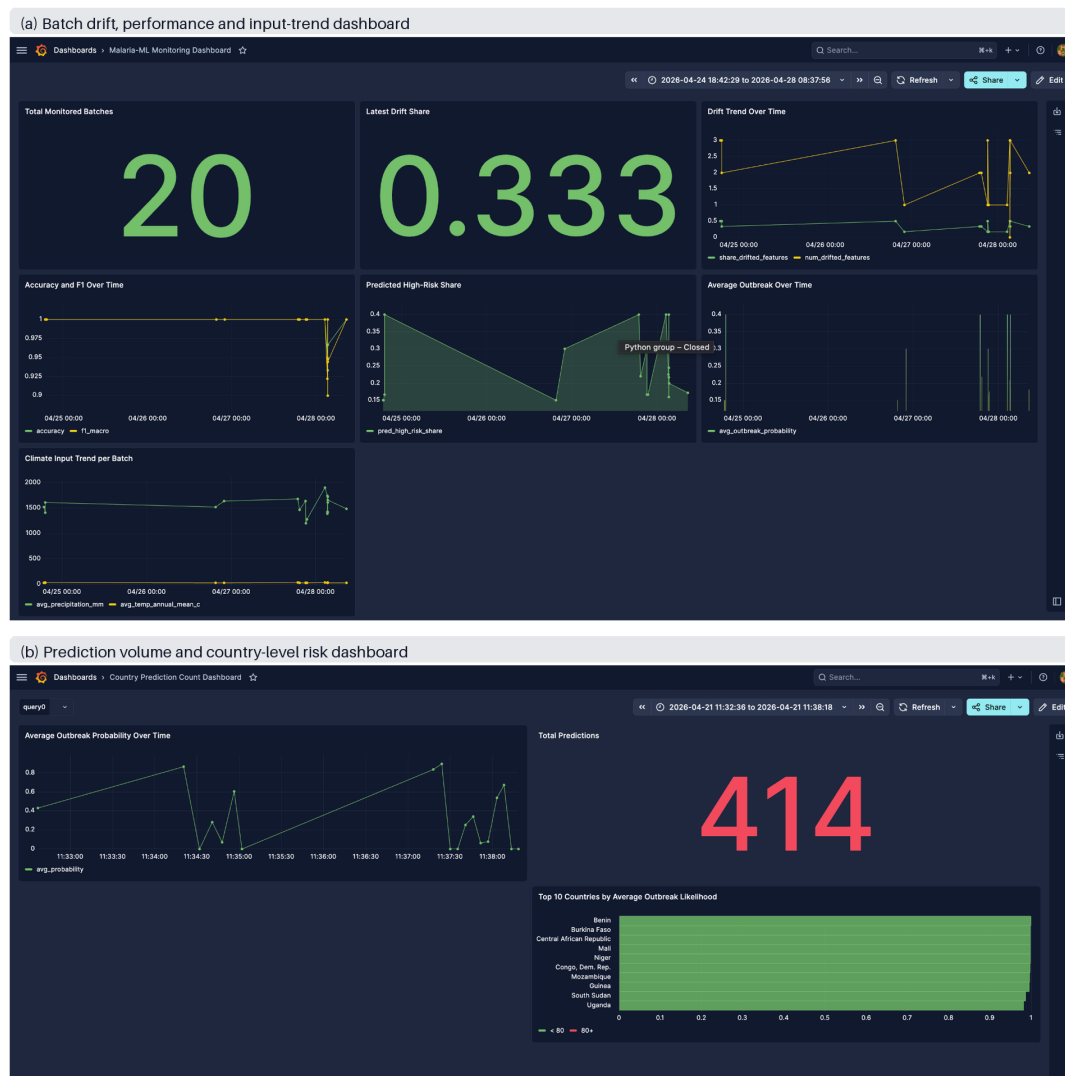


Fig. 4. Grafana monitoring evidence. The first dashboard summarizes 20 stored batches, drift share, batch metrics, prediction distribution, mean probability, and climate-input trends. The second shows prediction volume and countries with the highest average predicted risk.

remained useful for repeatable stress testing. Together, these findings show why target validity, shared offline/online transformations, statistically governed monitoring, resilient telemetry, and controlled model promotion are first-class MLOps concerns. MalOps is therefore best understood as a strong reproducible prototype and learning artifact rather than a field-ready malaria surveillance system.

ACKNOWLEDGMENT

The authors thank God, the course instructor, Dr Forooz Shahbazi Avarvand, and the providers of publicly available health and development data. The project was developed for the Artificial Intelligence module within the MSc IT, Digitalisation and Sustainability program at HSLU.

REPRODUCIBILITY, TOOLS, AND AI DISCLOSURE

The complete source code, configuration files, Docker assets, monitoring components, and report sources are available

in the project repository at:

https://github.com/Lamiorkor/Malaria_Outbreak_Prediction

The implementation was developed primarily in Python using FastAPI, scikit-learn, MLflow, Prefect, PostgreSQL, Grafana, Docker, GitHub Actions, and supporting scientific computing libraries including NumPy and pandas. Model experimentation, training, deployment, monitoring, and CI/CD execution were performed using the tools and infrastructure described throughout this report.

Both authors contributed to the design, implementation, testing, monitoring, documentation, and evaluation of the system. Responsibilities were shared throughout the project, with iterative review and refinement of code, experiments, architecture, and report content. Final decisions regarding implementation, analysis, and reporting were jointly validated by both authors.

Generative AI tools, including ChatGPT, Claude, and Google Gemini, were used during the project as development assistants. These tools supported code generation, debugging, explanation of concepts, architectural discussions, documentation drafting, report editing, and refinement of technical writing. In several stages of implementation, an iterative “vibe-coding” workflow was employed to accelerate prototyping and exploration of alternative solutions.

All generated code, configurations, workflows, and written material were reviewed, tested, modified, and validated by the authors before inclusion in the final system. The authors remained responsible for the design decisions, implementation choices, experimental results, interpretations, and conclusions presented in this report. AI assistance supported productivity and learning but did not replace the authors’ understanding of the system or the verification of reported outcomes.

REFERENCES

- [1] World Health Organization, “World malaria report 2025: Addressing the threat of antimalarial drug resistance,” World Health Organization, Geneva, Switzerland, 2025. [Online]. Available: <https://www.who.int/teams/global-malaria-programme/reports/world-malaria-report-2025>.
- [2] E. A. Mordecai, K. P. Paaijmans, L. R. Johnson, et al., “Optimal temperature for malaria transmission is dramatically lower than previously predicted,” *Ecology Letters*, vol. 16, no. 1, pp. 22–30, 2013. DOI: 10.1111/ele.12015.
- [3] L. L. M. Shapiro, S. A. Whitehead, and M. B. Thomas, “Quantifying the effects of temperature on mosquito and parasite traits that determine the transmission potential of human malaria,” *PLOS Biology*, vol. 15, no. 10, e2003489, 2017. DOI: 10.1371/journal.pbio.2003489.
- [4] D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [5] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, “The ml test score: A rubric for ml production readiness and technical debt reduction,” in *2017 IEEE International Conference on Big Data*, 2017, pp. 1123–1132. DOI: 10.1109/BigData.2017.8258038.
- [6] N. L. Boye and L. Kolli, *Malaria outbreak prediction: Project repository*, Accessed: 2026-06-06, 2026. [Online]. Available: https://github.com/Lamiorkor/Malaria_Outbreak_Prediction.
- [7] J. Wiens, S. Saria, M. Sendak, et al., “Do no harm: A roadmap for responsible machine learning for health care,” *Nature Medicine*, vol. 25, pp. 1337–1340, 2019. DOI: 10.1038/s41591-019-0548-6.
- [8] World Health Organization, “Ethics and governance of artificial intelligence for health: Who guidance,” World Health Organization, Geneva, Switzerland, 2021. [Online]. Available: <https://www.who.int/publications/item/9789240029200>.
- [9] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432.
- [10] B. V. Calster, D. J. McLernon, M. van Smeden, L. Wynants, and E. W. Steyerberg, “Calibration: The achilles heel of predictive analytics,” *BMC Medicine*, vol. 17, no. 230, 2019. DOI: 10.1186/s12916-019-1466-7.
- [11] D. R. Roberts, V. Bahn, S. Ciuti, et al., “Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure,” *Ecography*, vol. 40, no. 8, pp. 913–929, 2017. DOI: 10.1111/ecog.02881.
- [12] C. Huyen, *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. Sebastopol, CA, USA: O’Reilly Media, 2022.
- [13] S. Rabanser, S. Günnemann, and Z. C. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [14] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, 44:1–44:37, 2014. DOI: 10.1145/2523813.
- [15] World Bank, *Incidence of malaria (per 1,000 population at risk), indicator sh.mlr.incd.p3*, World Development Indicators, Accessed: 2026-06-06, 2026. [Online]. Available: <https://databank.worldbank.org/metadataglossary/world-development-indicators/series/SH.MLR.INCD.P3>.
- [16] World Bank, *Population density (people per sq. km of land area), indicator en.pop.dnst*, World Development Indicators, Accessed: 2026-06-06, 2026. [Online]. Available: <https://data.worldbank.org/indicator/EN.POP.DNST>.
- [17] World Bank, *Gdp per capita (constant 2015 us\$), indicator ny.gdp.pcap.kd*, World Development Indicators, Accessed: 2026-06-06, 2026. [Online]. Available: <https://data.worldbank.org/indicator/NY.GDP.PCAP.KD>.
- [18] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [19] MLflow Project, *MLflow documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://mlflow.org/docs/latest/>.
- [20] FastAPI Project, *Fastapi documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>.
- [21] Feast Project, *Feast documentation: Feature views and offline/online feature serving*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://docs.feast.dev/getting-started/concepts/feature-view>.
- [22] Celery Project, *Celery documentation: Task queues, brokers, and workers*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://docs.celeryq.dev/en/stable/getting-started/>.

- [23] Docker Inc., *Docker documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://docs.docker.com/>.
- [24] GitHub, *Github actions documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://docs.github.com/actions>.
- [25] Prefect Technologies, Inc., *Prefect documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://docs.prefect.io/>.
- [26] MLflow Project, *MLflow model registry documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://mlflow.org/docs/latest/ml/model-registry/>.
- [27] O. J. Dunn, "Multiple comparisons among means," *Journal of the American Statistical Association*, vol. 56, no. 293, pp. 52–64, 1961. DOI: 10.1080/01621459.1961.10482090.
- [28] N. Siddiqi, *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards*, 2nd ed. Hoboken, NJ, USA: Wiley, 2016. DOI: 10.1002/9781119282396.
- [29] V. M. Panaretos and Y. Zemel, "Statistical aspects of wasserstein distances," *Annual Review of Statistics and Its Application*, vol. 6, pp. 405–431, 2019. DOI: 10.1146/annurev-statistics-030718-104938.
- [30] Grafana Labs, *Grafana documentation*, Accessed: 2026-06-06, 2026. [Online]. Available: <https://grafana.com/docs/grafana/latest/>.
- [31] scikit-learn developers, *Model persistence: Security and maintainability limitations*, Accessed: 2026-06-06, 2026. [Online]. Available: https://scikit-learn.org/stable/model_persistence.html.

APPENDIX A

ADDITIONAL IMPLEMENTATION EVIDENCE

Figure 5 documents the hosted GitHub Actions run and GHCR image publication. Figure 6 shows both exploratory Logistic Regression variants and repeated final training runs. Figure 7 shows the two PostgreSQL tables used by the monitoring stack. Figure 8 documents completed Prefect flow runs.

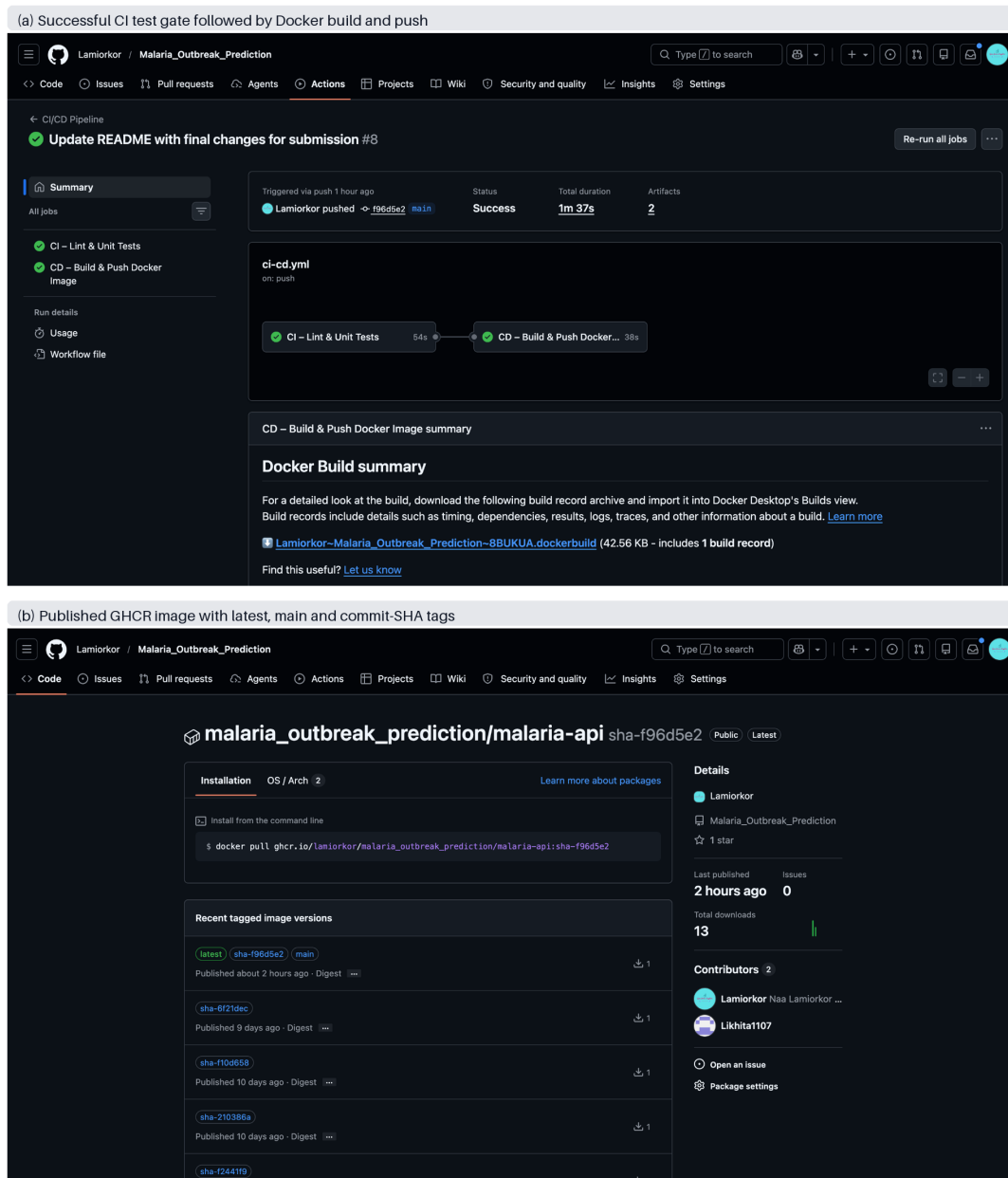


Fig. 5. CI/CD evidence: the successful test gate precedes Docker build and push, and the resulting GHCR package is tagged by branch, commit SHA, and latest. This is continuous delivery to a registry, not automatic rollout to a live cloud service.

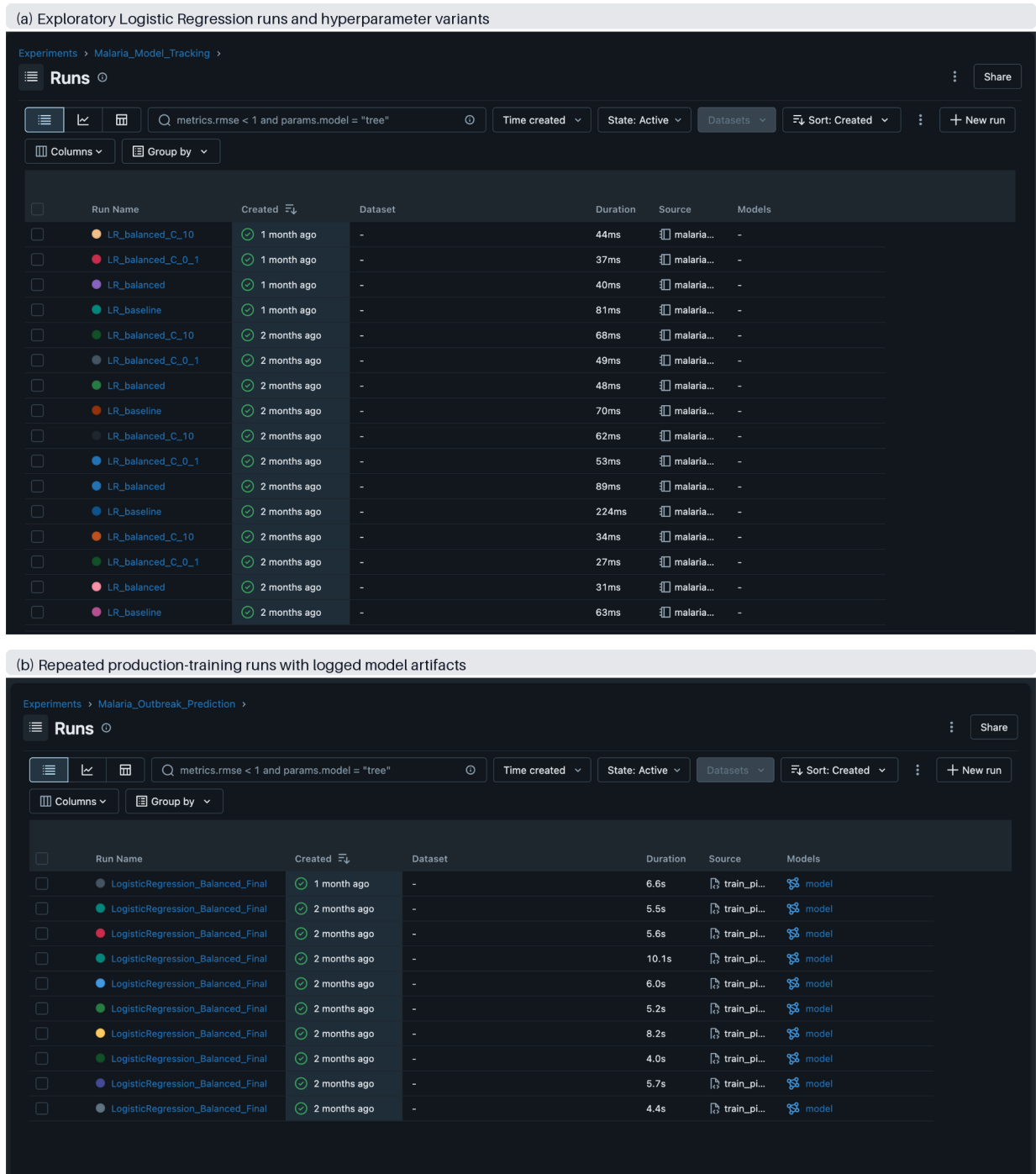


Fig. 6. MLflow evidence. Exploratory runs compared baseline, balanced, and regularization variants; the final training pipeline logged repeated LogisticRegression_Balanced_Final runs and model artifacts.

(a) Stored batch-level monitoring metrics

PostgreSQL > db > test > public > Select: monitoring_metrics

Select: monitoring_metrics

Select dataShow structureAlter tableNew item

SelectSearchSortLimitText lengthAction

50100[Save]

SELECT * FROM "monitoring_metrics" LIMIT 100 rows 0/1

idtimestampbatch_idbatch_sizedrift_mode

num_drifted_featuresshare_drifted_featuresaccuracyprecisionrecallf1_macro

avg_outbreak_probabilitypred_low_risk_sharepred_high_risk_shareavg_latency_msavg_malaria_incidenceavg_precipitation_mmax_avg_temp_annual_mean_c

ent	2026-04-24 16:42:29.211031	6d40d64	20	mixed	3	0.5	1	1	1	1	0.12062	0.85	0.15	7.845	63.69084	1521.96	26.173904046
ent	2026-04-24 16:53:43.454853	17773331	20	mixed	3	0.5	1	1	1	1	0.12062	0.85	0.15	11.831	63.69084	1521.96	26.173904046
ent	2026-04-24 17:06:42.807030	8b40d64	20	mixed	3	0.5	1	1	1	1	0.17945666666666667	0.8333333333333333	0.16666666666666667	6.258133333333333	78.90084	1407.0466666666667	24.177980055
ent	2026-04-24 17:07:03.752223	9169933	5	mixed	2	0.3333333333333333	1	1	1	1	0.40018	0.6	0.4	8.09	162.12096	1611.72	26.325333332
ent	2026-04-26 17:30:24.082627	43d37ac	20	mixed	3	0.5	1	1	1	1	0.12062	0.85	0.15	6.5835	63.69084	1521.96	26.173904046
ent	2026-04-26 18:56:26.262821	4224c637	10	extreme	1	0.16666666666666667	1	1	1	1	0.30042	0.7	0.3	8.423	129.9316	1643.72	26.245188433
ent	2026-04-27 16:50:28.86045	4e44126	5	extreme	2	0.3333333333333333	1	1	1	1	0.39966	0.6	0.4	8.866	150.112	1684.86	26.372333332
ent	2026-04-27 17:18:26.387967	130a264	50	mixed	2	0.3333333333333333	1	1	1	1	0.218224	0.76	0.22	6.286	83.180932	1467.894	24.975005
ent	2026-04-27 18:05:51.221811	85d40f1	10	extreme	1	0.16666666666666667	1	1	1	1	0.30042	0.7	0.3	6.405	129.9316	1643.72	26.245188433
ent	2026-04-27 18:08:44.26016	43d0113	20	extreme	3	0.5	1	1	1	1	0.16666666666666667	0.8333333333333333	0.16666666666666667	6.348333333333333	61.504	1199.126	24.71243889
ent	2026-04-27 19:25:04.534358	83a0c36	30	none	1	0.16666666666666667	1	1	1	1	0.17031333333333333	0.8333333333333333	0.16666666666666667	6.571333333333333	76.28633333333333	1279.133333333	23.37998005
ent	2026-04-28 00:31:29.870574	482a389	5	extreme	1	0.16666666666666667	1	1	1	1	0.40032	0.6	0.4	7.702	180.1344	1904.76	30.67243332
ent	2026-04-28 01:14:43.870793	59f46ac	40	extreme	1	0.5	0.86	0.7777777777777777	1	0.921875	0.20903	0.776	0.225	4.854	91.2276	1741.175	26.481247706
ent	2026-04-28 01:18:58.261232	345a3d2	25	mixed	3	0.5	1	1	1	1	0.115568	0.86	0.16	18.9468	70.95384	1422.88	24.805896966
ent	2026-04-28 01:19:20.632762	68f3272f	10	mixed	0	0	0	1	1	1	0.29942	0.7	0.3	6.964	116.9544	1395.84	24.545185833
ent	2026-04-28 01:19:34.263781	4679c794	60	mixed	0	0.9666666666666667	0.8461538461538461	1	0.9479166666666667	0.21303833333333333	0.7833333333333333	0.21666666666666667	6.427833333333333	81.207	1422.245	24.8871779166667	
ent	2026-04-28 01:20:16.540171	7247a023	5	mixed	2	0.3333333333333333	1	1	1	1	0.40032	0.6	0.4	7.625	162.12096	1611.72	26.325333332
ent	2026-04-28 01:20:45.61124	476a707e	45	extreme	3	0.5	0.9333333333333333	0.7272727272727272	1	0.8992870115567	0.22807177777777777	0.7555555555555556	0.24444444444444444	6.824444444444444	90.91893333333333	1724.9555555556	26.55891534444
ent	2026-04-28 01:22:24.771714	59f46ac5	30	extreme	3	0.5	0.9666666666666667	0.8333333333333333	1	0.94434172212801	0.38248	0.8	0.2	7.757333333333333	84.3436	1662.873333333	25.877980055
ent	2026-04-28 06:37:56.537787	c8d2632	35	mixed	2	0.3333333333333333	1	1	1	1	0.18151428571428571	0.8287142857142857	0.17142857142857143	3.973142857142857	76.99404571428571	1487.45142857143	24.59458254286

Whole result 20 rows

Modify[Save][Edit][Close][Delete]

Export (20)

Import

(b) Prediction logs with probabilities, model metadata and latency

PostgreSQL > db > test > public > Select: prediction_logs

Select: prediction_logs

Select data Show structure Alter table New item

Select Search Sort Limit Text length Action

50 100 [Reset]

SELECT * FROM "prediction_logs" LIMIT 100 rows 0/1

	id	country	year	temperature_mean	precipitation_mm	ndvi	gdp_per_capita	population_density	malaria_log1	malaria_log2	malaria_log3	outbreak_probability	outbreak_prediction	model_name	model_version	request_timestamp	inference_latency_ms	batch_id
ent	1	Ghana	2023	26.5	1200	MALI	2200	130	MALI	MALI	MALI	0.4307	0	0	0.0004	2026-04-21 09:32:36.814432	52.13	MALI
ent	2	Nigeria	2024	27.5	1500	MALI	2800	220	MALI	MALI	MALI	0.8688	0	0	0.0004	2026-04-21 09:34:26.814432	163.78	MALI
ent	3	South Africa	2024	9.5	900	MALI	8500	220	MALI	MALI	MALI	0	0	0	0.0004	2026-04-21 09:34:31.720146	25.08	MALI
ent	4	India	2024	25	1100	MALI	2500	450	MALI	MALI	MALI	0.2802	0	0	0.0004	2026-04-21 09:34:40.771665	23.32	MALI
ent	5	Brazil	2024	26	1800	MALI	9000	25	MALI	MALI	MALI	0.0682	0	0	0.0004	2026-04-21 09:34:47.735796	25.17	MALI
ent	6	Kenya	2024	24.5	1000	MALI	2000	95	MALI	MALI	MALI	0.0056	1	0	0.0004	2026-04-21 09:34:50.249724	26.69	MALI
ent	7	Norway	2024	5	800	MALI	9000	15	MALI	MALI	MALI	0	0	0	0.0004	2026-04-21 09:35:05.972611	29.25	MALI
ent	8	Nigeria	2023	27.2	1450	MALI	1950	220	MALI	MALI	MALI	0.8375	1	0	0.0004	2026-04-21 09:37:17.845207	155.18	MALI
ent	9	Nigeria	2025	27.8	1550	MALI	2050	225	MALI	MALI	MALI	0.8956	1	0	0.0004	2026-04-21 09:37:23.940617	35.7	MALI
ent	10	Switzerland	2023	9.2	850	MALI	8400	220	MALI	MALI	MALI	0	0	0	0.0004	2026-04-21 09:37:28.969811	26.82	MALI
ent	11	Switzerland	2025	10	920	MALI	8600	225	MALI	MALI	MALI	0	0	0	0.0004	2026-04-21 09:37:35.145161	22.1	MALI
ent	12	India	2023	24.7	1050	MALI	2400	445	MALI	MALI	MALI	0.2517	0	0	0.0004	2026-04-21 09:37:40.684138	20.87	MALI
ent	13	India	2025	25.5	1150	MALI	2600	460	MALI	MALI	MALI	0.3401	0	0	0.0004	2026-04-21 09:37:46.182021	28.06	MALI
ent	14	Brazil	2023	28.8	1750	MALI	8800	24	MALI	MALI	MALI	0.0809	0	0	0.0004	2026-04-21 09:37:51.515625	23.88	MALI
ent	15	Brazil	2025	26.3	1850	MALI	9200	26	MALI	MALI	MALI	0.0767	0	0	0.0004	2026-04-21 09:37:56.845019	38.46	MALI
ent	16	Kenya	2023	24.2	950	MALI	2100	92	MALI	MALI	MALI	0.1593	1	0	0.0004	2026-04-21 09:38:03.045382	30.44	MALI
ent	17	Kenya	2025	25	1050	MALI	2300	100	MALI	MALI	MALI	0.1713	1	0	0.0004	2026-04-21 09:38:08.192919	23.22	MALI
ent	18	Norway	2023	4.5	780	MALI	8800	15	MALI	MALI	MALI	0	0	0	0.0004	2026-04-21 09:38:13.389023	28.94	MALI
ent	19	Norway	2025	5.5	820	MALI	9100	16	MALI	MALI	MALI	0	0	0	0.0004	2026-04-21 09:38:18.427259	23.92	MALI
ent	20	Canada	2023	26.3	1050	MALI	10000	50	MALI	MALI	MALI	0.0948	0	0	0.0004	2026-04-24 16:10:16.024795	856.46	MALI
ent	21	Ghana	2023	26.5	1200	MALI	2200	130	MALI	MALI	MALI	0.4307	0	0	0.0004	2026-04-26 17:26:23.171286	72.19	MALI
ent	22	Ghana	2023	26.5	1200	MALI	2200	130	MALI	MALI	MALI	0.4307	0	0	0.0004	2026-04-26 18:48:01.071377	172.97	MALI
ent	23	Ghana	2023	26.5	1200	MALI	2200	130	MALI	MALI	MALI	0.4307	0	0	0.0004	2026-04-26 18:49:26.967326	111.85	MALI
ent	24	Ghana	2023	26.5	1200	MALI	2200	130	MALI	MALI	MALI	0.4307	0	0	0.0004	2026-04-26 19:03:16.558349	131.8	MALI
ent	25	Benin	2020	30.71333333	1305.7	MALI	788.6576287	115.913959	399.91	407.71	415.74	0.9999	1	0	0.0004	2026-04-26 19:56:26.229543	20.22	622a2687
ent	26	Eswatini	2021	27.7838	1624.4	MALI	2603.3246674	70.1507581	0.7	0.76	2.1	0.0014	0	0	0.0004	2026-04-26 19:56:26.2358	6.33	622a2687
ent	27	Kenya	2020	26.60775	1122.4	MALI	528.48579715	108.970909	23.81	26.44	79.71	0.0009	0	0	0.0004	2026-04-26 19:56:26.257001	7.18	622a2687
ent	28	South Africa	2020	27.3110833	3204.4	MALI	3804.0873812	161.979789	0.19	0.38	0.39	0.0006	0	0	0.0004	2026-04-26 19:56:26.2572925	10.25	622a2687
ent	29	Guyana	2020	29.6408333	3103.1	MALI	6307.4270035	3.824386663	32.25	38.87	32.75	0.0008	0	0	0.0004	2026-04-26 19:56:26.259582	7.37	622a2687
ent	30	Mali	2021	32.48266667	366.6	MALI	610.63275658	18.34847852	349.39	325.76	377.17	0.9997	1	0	0.0004	2026-04-26 19:56:26.260131	6.47	622a2687
ent	31	Myanmar	2020	27.4986	2718.1	MALI	918.9197046	81.2301129	1.71	2.39	2.67	0.0003	0	0	0.0004	2026-04-26 19:56:26.260718	7.2	622a2687
ent	32	South Sudan	2021	28.45104167	1170	MALI	785.1030302	17.1458454	281.6	285.93	293.3	0.0012	1	0	0.0004	2026-04-26 19:56:26.26126	6.46	622a2687
ent	33	Tajikistan	2020	6.9785	898.3	MALI	877.8499972	71.8128836	0	0	0	0.0016	0	0	0.0004	2026-04-26 19:56:26.261535	6.95	622a2687
ent	34	Thailand	2021	30.02525	2108.0	MALI	4285.0688755	140.3568212	0.23	0.3	0.38	0.0005	0	0	0.0004	2026-04-26 19:56:26.261975	6.2	622a2687
ent	35	Benin	2020	29.41333333	1194.85	MALI	1126.654041	115.913959	399.91	407.71	415.74	0.9996	1	0	0.0004	2026-04-27 16:50:28.888206	9.24	a6944126
ent	36	Guatemala	2020	26.91108333	2205.4	MALI	4005.859116	161.979789	0.19	0.39	0.39	0.0006	0	0	0.0004	2026-04-27 16:50:28.892472	6.17	a6944126
ent	37	Guyana	2020	28.3048333	2745.0	MALI	9010.611005	3.824386663	32.25	38.87	32.75	0.0005	0	0	0.0004	2026-04-27 16:50:28.892968	6.17	a6944126
ent	38	Mali	2021	31.18266667	324.3	MALI	872.3325094	18.34847852	349.39	325.76	377.17	0.9984	1	0	0.0004	2026-04-27 16:50:28.893443	6.03	a6944126
ent	39	Thailand	2021	28.72523	1863.3	MALI	6121.526965	140.3568212	0.23	0.3	0.38	0.0005	0	0	0.0004	2026-04-27 16:50:28.893949	6.86	a6944126
ent	40	Azerbaijan	2020	13.7579833	491.7	MALI	6460.5002992	12.1214768	0	0	0	0	0	0	0.0004	2026-04-27 17:19:28.366024	5.86	1306264
ent	41	Benin	2020	29.6133333	1142.8	MALI	1536.521772	115.913959	399.91	407.71	415.74	0.9997	1	0	0.0004	2026-04-27 17:19:28.374442	6.19	1306264
ent	42	Benin	2020	29.6133333	1142.8	MALI	1536.521772	115.913959	399.91	407.71	415.74	0.9997	1	0	0.0004	2026-04-27 17:19:28.374442	6.19	1306264
ent	43	Uthmaniyah	2021	13.7525	2420	MALI	789.707564	20.3146303	0.3	0	0.001	0.0007	0	0	0.0004	2026-04-27 17:19:28.374597	6.07	1306264
ent	44	Bolivia	2020	22.92416667	1260.6	MALI	2394.8830506	10.9658954	2.39	1.39	1.2	0.001	0	0	0.0004	2026-04-27 17:19:28.375189	7.67	1306264
ent	45	Brazil	2021	27.61346667	1937.1	MALI	8596.26017832	25.0716273	3.95	4.24	5.23	0.0004	0	0	0.0004	2026-04-27 17:19:28.376231	6.89	1306264
ent	46	Cabo Verde	2020	26.37	2051	MALI	2095.6141368	10.9658954	0.21	0.21	0.21	0.0004	0	0	0.0004	2026-04-27 17:19:28.376369	6.33	1306264
ent	47	Cabo Verde	2020	26.376	2058	MALI	3199.92951464	128.207444	0	0	0.001	0.0015	0	0	0.0004	2026-04-27 17:19:28.377102	6.13	1306264
ent	48	Chad	2021	27.8925	354.2	MALI	78.52420152	14.1598412	208.74	212.22	224.99	0.8007	1	0	0.0004	2026-04-27 17:19:28.377499	6.25	1306264
ent	49	Congo, Dem. Rep.	2021	25.8638	1697.3	MALI	446.34795448	43.477956	0.382	0.39	0.47	0.9979	1	0	0.0004	2026-04-27 17:19:28.377685	6.81	1306264
ent	50	Democratic Republic of the Congo	2021	27.64883333	1551	MALI	7673.23573548	236.7669998	1.6	0.25	0.002	0.0005	0	0	0.0004	2026-04-27 17:19:28.378144	7.27	1306264

Load more data

Page: 12,245 - 9

Whole result

Modify

Sell: 100 (0)

Export (414)

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 - 9

12,245 -

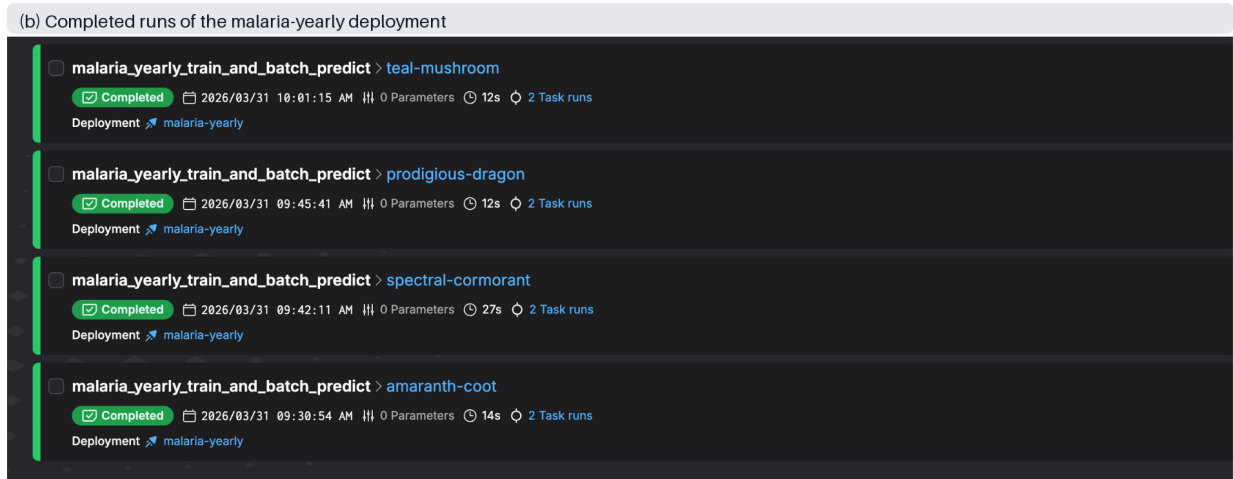
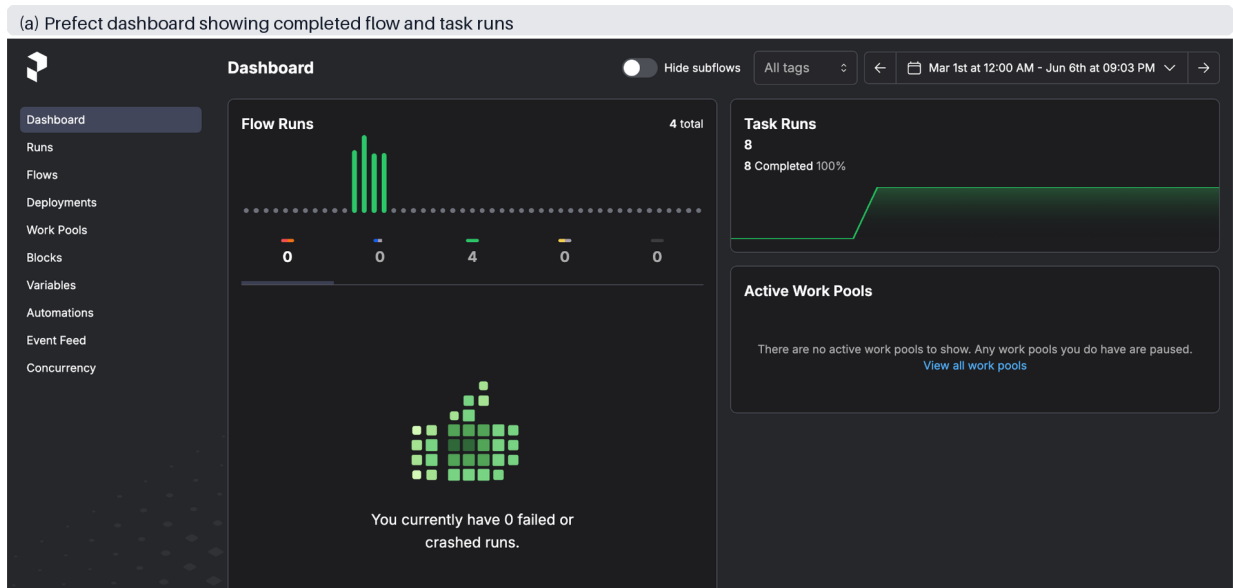


Fig. 8. Prefect evidence. The dashboard records completed flow and task runs, and the `malaria-yearly` deployment produced four successful executions during the implementation demonstration.